

Event-driven execution

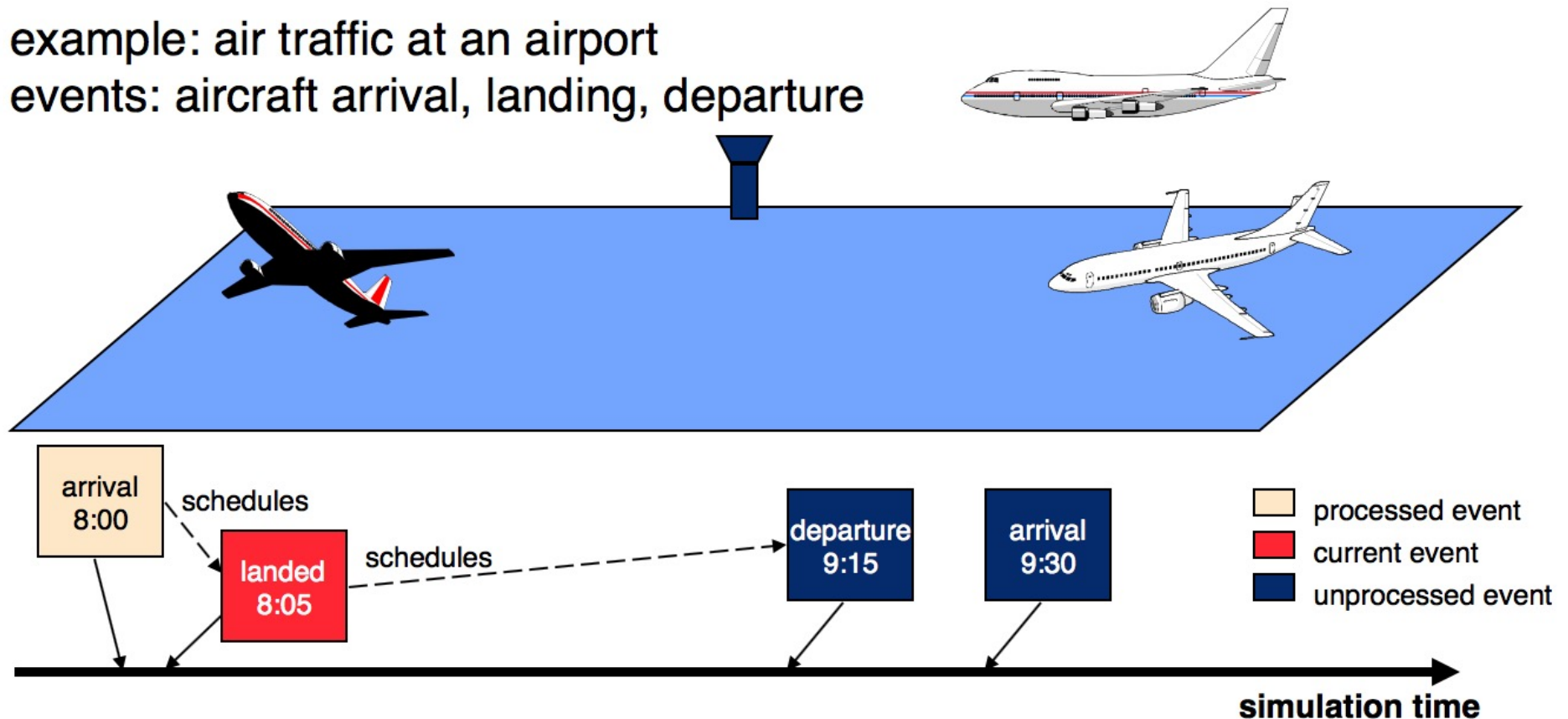
- Rather than compute a new value for state variables each time step, it may be more efficient to only update the variables when "*something interesting*" occurs
- The "something interesting" that occurs is referred to as an *event*
- This is the key idea behind *discrete event simulations (DES)*
- An event is an abstraction used in the simulation to model some instantaneous action in the physical system
- **Each event has a time stamp** associated with it that indicates the point in simulation time when the event occurs
- Each event usually results in some change in one or more state variables defined by the simulation

Example event-driven simulation

- Consider a simulation of an aircraft flying from New York to Los Angeles
- A time-stepped simulation with time step size of 10 minutes might compute the aircraft's new position every 10 minutes
- An event-driven approach is more efficient if intermediate positions are not needed
 - only update the aircraft's position variable when simulation time advances to the time the aircraft reaches Los Angeles
 - the aircraft arriving in Los Angeles is modeled by an event
 - changes in state variables only occur as the result of some event

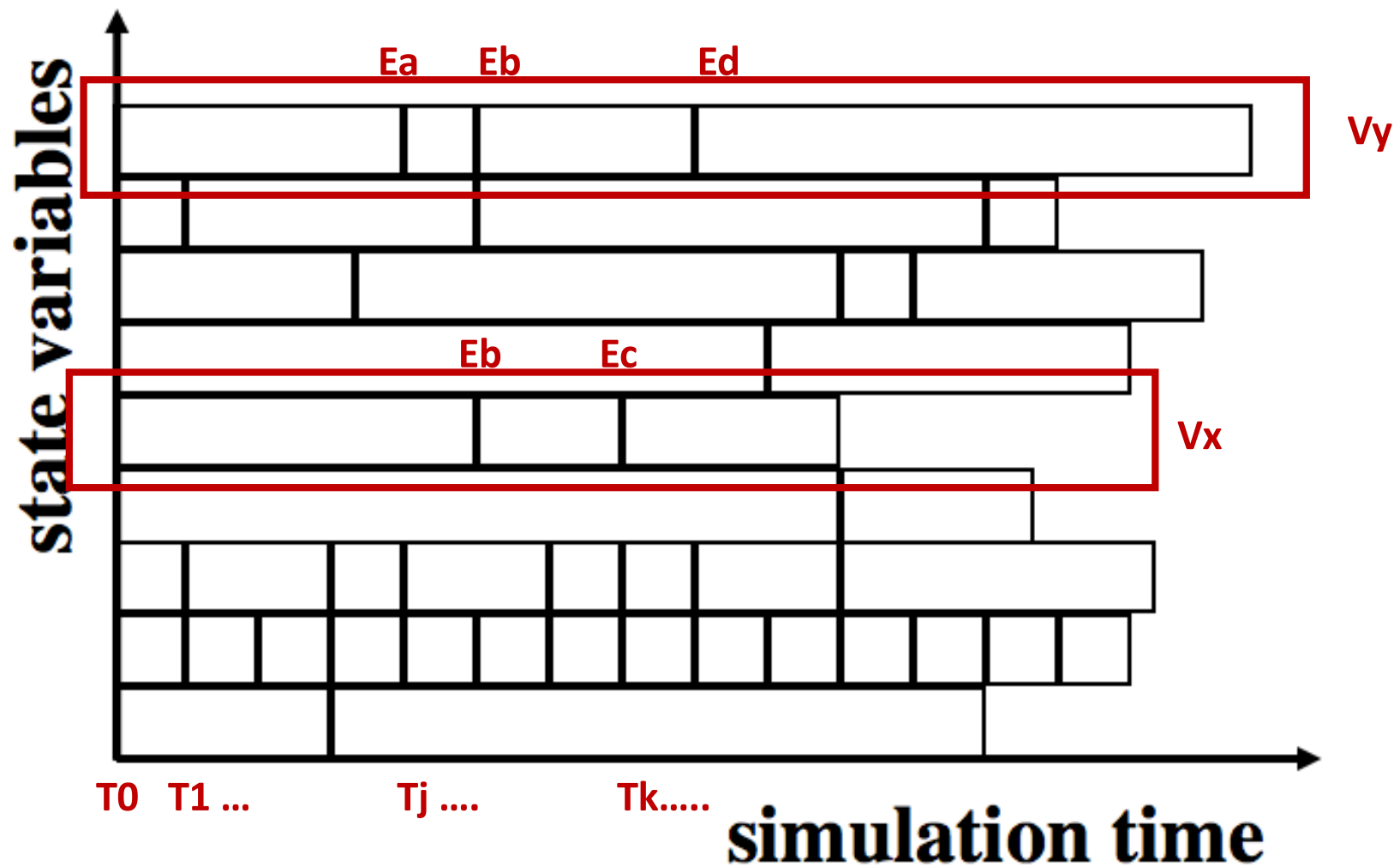
Example event-driven simulation

example: air traffic at an airport
events: aircraft arrival, landing, departure



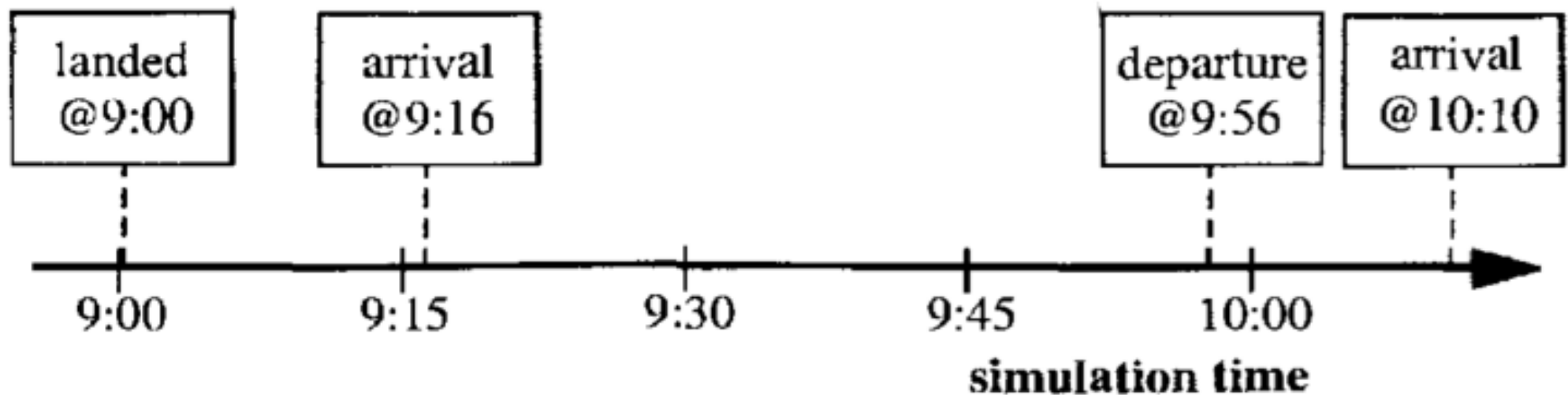
- Unprocessed events are stored in a pending event list
- Events are processed in time stamp order

Space-time diagram for event-driven simulation



Example sequence of events

- Simulation time advances from the time stamp of one event to the next
- From a computational standpoint, the simulation can be viewed as a sequence of computations, one for each event, transforming the system across simulated time in a manner representing the behavior of the actual system

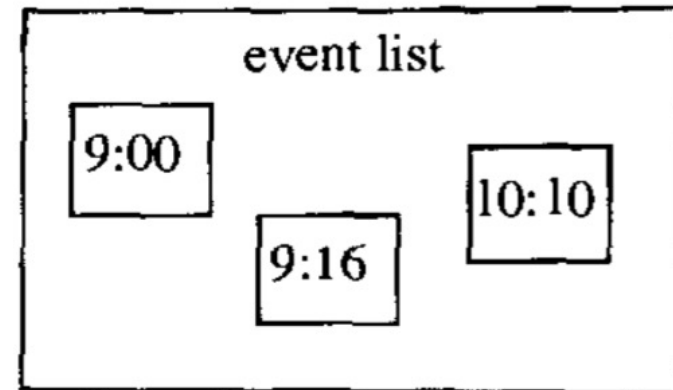
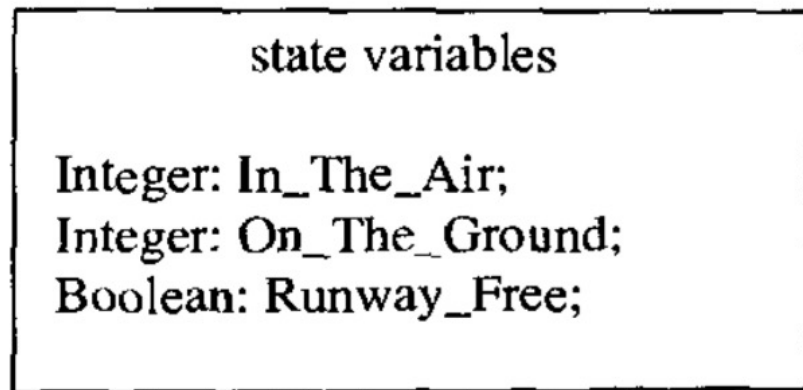


Event-driven and real-time simulation

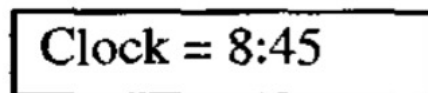
- A variation of the event-driven paradigm can also be used for real-time simulations
- One approach is to prevent simulation time from advancing to the time stamp of the next event until wallclock time has advanced to the time of this event
- In other words, if the time stamp of the next event is T_s simulation time is not advanced to T_s until $W2S(T_w)$ reaches T_s where T_w is the current value of wallclock time

DES programs

- A *sequential* discrete event simulation program typically utilizes three data structures:
 - The *state variables* that describe the state of the system
 - An *event list* containing events that are to occur at some simulation time
 - A *global clock* variable to denote the instant on the **simulation time** axis at which the simulation now resides

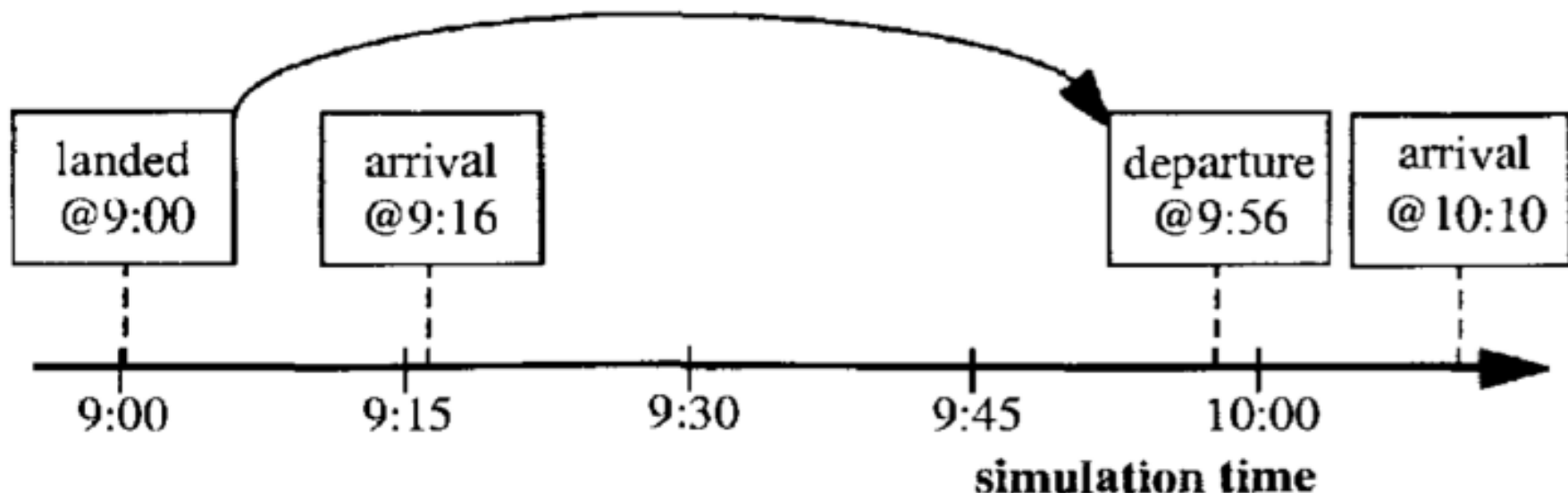


Clock = 8:45

A rectangular box with a black border. Inside, the text "Clock = 8:45" is centered.

Event scheduling

- In the physical system, "events" such as an aircraft arrival "just happen"
- In the simulated world, nothing happens unless the simulation computation makes it happen
- In other words, a mechanism is required to create new events
- The mechanism for creating a new event in the simulation is called *scheduling an event*



Event *scheduling*

- "*Scheduling an event*" in the simulator is implemented by:
 - allocating memory for a new event
 - filling in the fields for the time stamp, event type, and the associated parameters, and
 - adding the event to the event list data structure
- Event scheduling is one way that simulation programs can model causal relationships in the physical system

Architecture of a DES program

model of the
physical
system

Simulation Application

- state variables
- code modeling system behavior
- I/O and user interface software

calls to
schedule
events

calls to event
handlers

independent
of the
simulation
application

Simulation Executive

- event list management
- managing advances in simulation time

Event-oriented program

Event handler procedures

state variables

```
Integer: InTheAir;  
Integer: OnTheGround;  
Boolean: RunwayFree;
```

Arrival
Event

```
{  
    ...  
}
```

Landed
Event

```
{  
    ...  
}
```

Departure
Event

```
{  
    ...  
}
```

Simulation application

Simulation executive

Now = 8:45

Pending Event List (PEL)

9:00

9:16

10:10

Event processing loop

```
While (simulation not finished)  
    E = smallest time stamp event in PEL  
    Remove E from PEL  
    Now := time stamp of E  
    call event handler procedure
```

Simulation executive *main loop*

```
while (simulation is in progress)
    remove smallest time stamped event from
        event list
    set simulation time clock to time stamp of
        this event
    execute event handler in application to
        process event
```

- The event handler may do two things:
 1. modify state variables to model changes in the state of the physical system that result from this event
 2. schedule new events into the simulated future

Causality properties

- There are two important issues in a discrete event simulation that are worth highlighting, because they will become very important when we consider execution on parallel and distributed computers
 1. the **simulation application** can only schedule events into the simulated future, namely the time stamp of any new event must be at least as large as the current time of the simulation
 2. the **simulation executive** always processes the event containing the smallest time stamp next
- Both relate to ensuring that the simulation faithfully reproduces **causal relationships** in the physical system

DES program implementation

- A DES can in principle be executed by “hand”, but is usually implemented as a computer program
- Different types of DES implementations, with different characteristics in terms of flexibility and cost
 - use of a **general-purpose programming language (GPL)**, such as Java or C++ (highest flexibility, but requires a substantial technical know-how and a considerable extra development effort)
 - use of a **simulation package** (or **SPL – simulation programming language**) that provides the simulation executive with synchronization and communication mechanisms, such as Simjava or CSIM (loses some flexibility, while gaining a more cost-effective development)
 - use of a **high-level simulation tool**, such as Arena or SIMUL8 (no flexibility, best cost-effective compromise)

“Hand” DES Example

- *Single-server queue*

- **Objective**

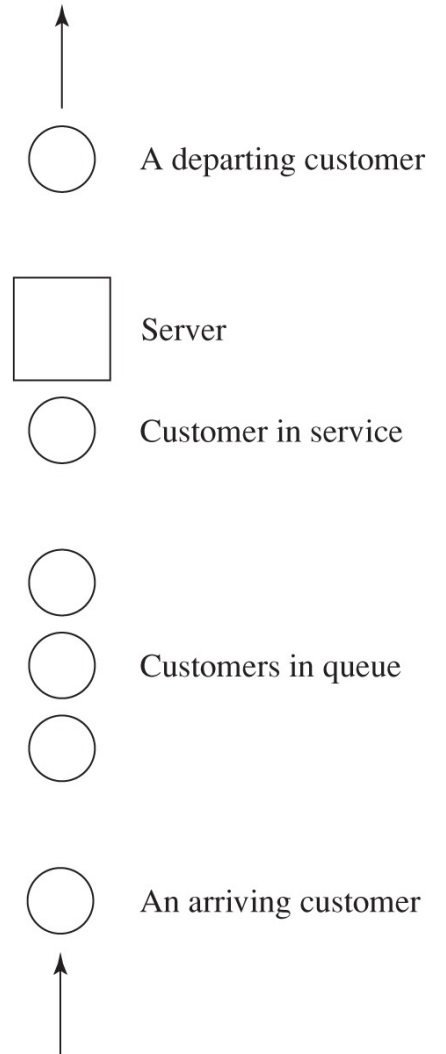
- Estimate expected average delay in queue (line, not service)

- **State variables**

- Status of server (idle, busy) – needed to decide what to do with an arrival
 - Current length of the queue – to know where to store an arrival that must wait in line
 - Time of arrival of each customer now in queue – needed to compute time in queue when service starts

- **Events**

- Arrival of a new customer
 - Service completion (and departure) of a customer
 - Maybe – end-simulation event (a “fake” event) – whether this is an event depends on how simulation terminates (a modeling decision)



“Hand” DES Example

- Next-event time advance for the single-server queue

t_i = time of arrival of i th customer ($t_0 = 0$)

$A_i = t_i - t_{i-1}$ = inter-arrival time between $(i-1)$ st and i th customers (usually assumed to be an IID random variable)

S_i = service-time requirement of i th customer (another IID random variable)

D_i = delay in queue of i th customer

$c_i = t_i + D_i + S_i$ = time i th customer completes service and departs

e_j = time of occurrence of the j th event (of any type), $j = 1, 2, 3, \dots$

- Possible trace of events

